面向端侧设备的大模型知识蒸馏技术解析

中文标题：面向端侧设备的大模型知识蒸馏技术解析

English Title: Knowledge Distillation for Large Language Models on Edge Devices

资源链接

- **arXiv**: https://arxiv.org/abs/2603.13765[1]
- **GitHub**: https://github.com/AlejandroParedesLT/knowledge_distillLLM[2]

一句话亮点本文提出了一个结合知识蒸馏（KD）与链式思考（CoT）强化学习的框架，旨在将大模型（LLM）高效压缩，使其适用于资源受限的端侧设备，同时在特定任务（如编程）上保持强大的推理能力。[3]

核心创新

本文的核心创新在于其提出并验证了一套**“蒸馏-增强-量化”**的完整流程，旨在打造轻量级、高性能且可部署于端侧的专用大模型。

1. **多领域知识蒸馏框架**
   - **方法**：采用标准的知识蒸馏（Knowledge Distillation, KD）方法，将一个大型的“教师模型”（如 Qwen-3B）的知识迁移到一个小型的“学生模型”（如 Qwen-0.5B）。该过程通过最小化教师和学生模型输出概率分布之间的 KL 散度来实现。
   - **创新点**：系统性地在**英语、西班牙语和编程代码**三个不同领域验证了知识蒸馏的有效性。实验表明，在模型对特定领域先验知识有限的情况下（如西班牙语），知识蒸馏比传统的监督微调（SFT）能更快地达到更高性能，证明了其在低资源场景下的迁移学习优势。
2. **结合链式思考（CoT）的强化学习增强**
   - **方法**：针对复杂的编程任务，在知识蒸馏的基础上，引入了基于“群体相对策略优化”（Group Relative Policy Optimization, GRPO）的强化学习（RL）流程。该方法利用带有链式思考（CoT）标注的数据集进行训练，激励模型生成更长、更具逻辑性的推理步骤。[4]
   - **创新点**：将 CoT 与 RLHF 技术（特别是 GRPO 算法）结合，用于**增强小型专业模型**（而非训练通用基础模型）的复杂推理能力。实验证明，这种方法能显著提升模型在代码生成任务上的正确性和逻辑连贯性，使其能够解决更复杂的问题。
3. **面向端侧部署的量化压缩**

- **方法**：在蒸馏和强化学习之后，对学生模型进行“训练后量化”（Post-Training Quantization, PTQ），具体采用 4-bit 权重量化（W4）。[5]
- **创新点**：完成了从模型压缩到**实际部署**的闭环。通过量化，模型的内存占用和推理延迟被进一步降低，使其达到了可以在资源受限的边缘设备（on-device）上运行的水平，为大模型在移动端、嵌入式系统等场景的落地提供了实践路径。

应用场景

该研究成果主要面向以下场景：

- **端侧 AI 应用**：在智能手机、笔记本电脑、智能汽车、物联网设备等算力与内存有限的硬件上，部署具备特定能力的 AI 模型。例如，实现离线的代码助手、语法检查工具或多语言翻译功能。
- **低成本专用模型定制**：对于需要特定领域（如金融、法律、医疗）知识，但预算有限的中小型企业或研究机构，可以通过此方法，将通用大模型的能力蒸馏到小型专用模型中，以较低的成本获得高性能的解决方案。
- **隐私保护计算**：由于模型和数据均可在本地运行和处理，无需上传至云端，因此可以满足对数据隐私有严格要求的应用场景，如个人健康助手、本地文档分析等。

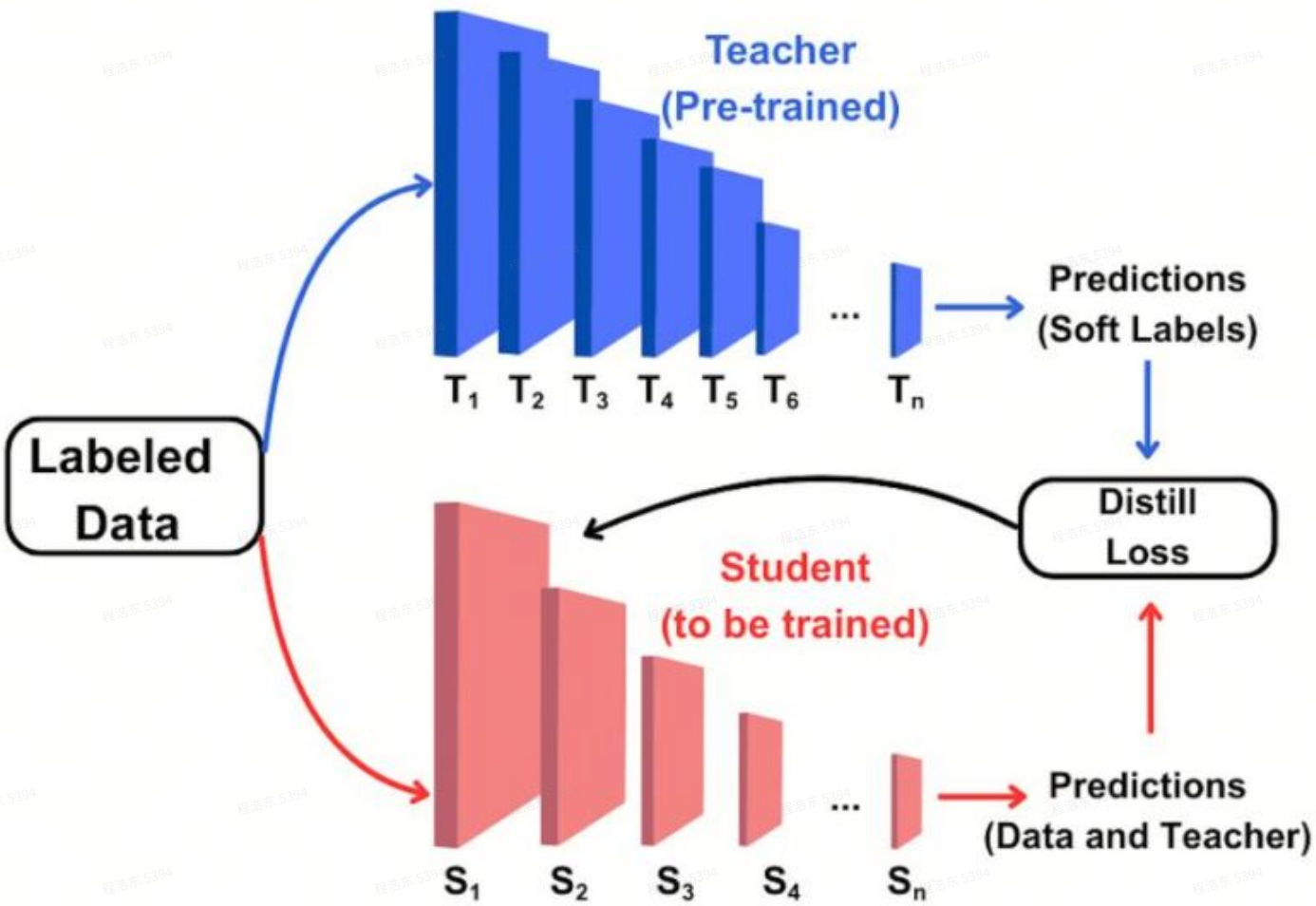

图 1：知识蒸馏流程示意图，展示了教师模型如何指导学生模型进行学习。

研究方法拆解

模型结构

该研究的核心是基于 **Transformer** 架构的“教师-学生”模型。

- **教师模型 (Teacher Model)**：一个预训练好的、参数量较大的语言模型，例如 **Qwen-3B** 或 **Qwen-1.5B**。它拥有丰富的知识和强大的能力，作为知识的来源。
- **学生模型 (Student Model)**：一个参数量较小、结构与教师模型相似（但层数或维度更低）的语言模型，例如 **Qwen-0.5B**。它的目标是通过学习来模仿教师模型的行为，从而以更小的体积实现接近教师的性能。[6]

两个模型均为 **Decoder-only** 的自回归语言模型，其基础构建模块是标准的 Transformer Block，包含多头自注意力机制（Multi-Head Self-Attention）和前馈神经网络（Feed-Forward Network）。

关键公式与推导

1. 知识蒸馏 (Knowledge Distillation, KD)知识蒸馏的核心思想是让学生模型学习教师模型的软目标（soft targets），即教师模型输出的完整概率分布，而不仅仅是概率最高的那个词（硬目标）。

- **目标函数 (L_KD)**：学生模型通过最小化与教师模型输出分布的交叉熵来学习。对于一个给定的上下文 `x<\_t`，目标函数定义为：$L_{KD} = -\sum_{t=1}^{T}\sum_{v\in V} P_T(v|x_{<t}) \log P_S(v|x_{<t})$
  - `T` 是序列长度。
  - `V` 是词汇表。
  - `P_T(v|x<\_t)` 是教师模型在上下文 `x<\_t` 后，预测下一个词为 `v` 的概率。
  - `P_S(v|x<\_t)` 是学生模型在相同上下文下，预测下一个词为 `v` 的概率。

  这个公式等价于最小化教师和学生模型分布之间的 **KL 散度 (Kullback-Leibler Divergence)**，因为它忽略了不依赖于学生模型参数的教师模型熵。

2. 链式思考的强化学习 (RL with CoT)为了增强模型的推理能力，研究中使用了 **群体相对策略优化 (Group Relative Policy Optimization, GRPO)** 算法。

- **GRPO 目标函数 (J_GRPO)**：GRPO 是一种无评论家（critic-free）的 RL 算法，它通过比较一组生成结果的好坏来更新策略。其目标函数定义为：

$$J_{GRPO}(\theta) = \mathbb{E}_{q\sim P(Q),\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{old}}(O|q)}\left[\frac{1}{G}\sum_{i=1}^{G}\left(\min(r_iA_i, \text{clip}(r_i, 1-\epsilon, 1+\epsilon)A_i)\right)\right] - \beta D_{KL}(\pi_\theta\|\pi_{ref})$$

  - `G` 是每组采样输出的数量。
  - `r_i` 是第 `i` 个输出的奖励分数。

- `A_i` 是通过对组内奖励进行归一化计算出的**优势 (Advantage)**：
  $$A_i = \frac{r_i - \text{mean}(\{r_1, ..., r_G\})}{\text{std}(\{r_1, ..., r_G\})}$$
- `β` 是 KL 散度惩罚项的权重，用于防止模型偏离参考策略 `π_ref` 太远。

损失函数

- **知识蒸馏损失**：如上所述，即学生模型与教师模型输出分布之间的交叉熵损失。这是模型压缩阶段的主要损失函数。
- **强化学习损失**：在 CoT 增强阶段，使用 GRPO 目标函数作为损失函数。这里的“损失”是要最大化的奖励函数，通过梯度上升进行优化。奖励函数 `r_i` 根据生成文本是否包含正确的推理步骤（如 "Step 1", "Step 2"）、是否符合 CoT 格式（如 `<think>...</think>`）以及最终答案的正确性来设计。

特征选择与预处理

- **数据来源**：
  - **通用问答**：Dolly-15k 数据集（英语和西班牙语版本），包含由人类生成的指令和回复。
  - **代码生成与修复**：
    - **BugNet**：包含错误代码、错误信息和修复后正确代码的三元组。
    - **PyTorrent**：高质量的 Python 代码库中提取的自然语言描述与对应代码片段对。
  - **链式思考**：`codeforces-cots` 数据集，包含来自 Codeforces 编程竞赛平台、经过 CoT 标注的问题和解决方案。
- **预处理**：数据被处理成标准的“指令-回复”格式。对于知识蒸馏，教师模型首先对指令生成回复，形成蒸馏所需的目标分布。对于 RL 训练，数据被格式化以适应 CoT 的提示结构。Tokenization 使用模型（如 Qwen）自带的分词器。

训练策略与优化算法

- **优化器**：未明确提及，但通常在此类任务中使用 AdamW。
- **学习率**：在知识蒸馏阶段，对学习率进行了实验，最终发现 `5e-4` 是一个在多个任务上表现稳健的选择。
- **并行策略**：训练利用了 **DeepSpeed** 框架，支持模型并行和数据并行，以在多 GPU 环境下高效训练。
- **混合精度**：虽未明确提及，但 DeepSpeed 通常默认启用混合精度训练（如 BF16），以加速计算并减少内存占用。
- **训练流程**：
  a. **教师模型准备**：如果需要，首先对教师模型在特定领域数据上进行 SFT（如在西班牙语 Dolly 数据集上微调 Qwen-3B）。

b. **知识蒸馏**：使用教师模型生成的软目标来训练学生模型。

c. **RL 增强（可选）**：对于需要复杂推理的任务，在蒸馏后的学生模型上，使用 GRPO 算法和 CoT 数据集进行强化学习。

d. **量化**：对最终的学生模型应用 4-bit PTQ。

评估标准

- **数据集**：评估在对应任务的测试集上进行，如 Dolly、BugNet 和 PyTorrent 的保留部分。
- **指标**：
  - 主要使用 **ROUGE-L** 分数来衡量生成文本与参考答案在内容上的重叠程度，特别是在摘要和翻译等任务中。
  - 对于代码任务，除了 ROUGE-L，还通过人工检查和案例分析来评估生成代码的**正确性**和**逻辑合理性**。
- **基线对比**：
  - **Base Model**：未经任何微调的学生模型。
  - **SFT Model**：仅使用监督微调训练的学生模型。
  - **KD Model**：使用知识蒸馏训练的学生模型。
  - **Teacher Model**：作为性能上限的教师模型。通过对比这些模型在相同评估集上的表现，来验证 KD 和 RL-CoT 的有效性。

实验重复性与可复现性

- **开源代码**：项目在 GitHub 上开源，提供了完整的代码库，包括训练、评估和数据处理的脚本。[2]
- **代码结构**：仓库结构清晰，分为 `configs`，`data_utils`，`minillm`，`scripts` 等目录，方便复现。
- **运行脚本**：提供了 `.sh` 脚本（如 `sft_xlarge_pytorrent.sh`，`kd_base_bugnet.sh`），封装了启动训练所需的命令和参数，降低了复现门槛。
- **依赖与环境**：`README` 文件中明确列出了所需的主要依赖，如 PyTorch, DeepSpeed, Transformers, PEFT 等，并提供了 `install.sh` 脚本来辅助环境搭建。
- **可复现实践建议**：要复现实验，建议遵循以下步骤：

  a. 克隆 GitHub 仓库。

  b. 使用 `install.sh` 或手动安装所需依赖。

  c. 根据 `README` 中的数据结构说明准备数据集。

  d. 执行 `scripts` 目录下的相应脚本来启动训练或评估。

假设与限制

- **假设**：
  - 假设学生模型与教师模型架构相似，能够有效学习教师模型的内部知识表示。
  - 假设 GRPO 中组内奖励的相对排序足以提供有效的优化信号，从而无需学习一个显式的价值函数。
- **限制与潜在风险**：
  - **知识蒸馏的上限**：学生模型的性能上限受限于教师模型。如果教师模型在某个领域表现不佳，学生模型很难超越它。
  - **任务泛化性**：尽管在多个领域进行了测试，但通过蒸馏获得的专用模型可能在训练领域之外的任务上表现不佳，泛化能力弱于通用大模型。
  - **RL 训练的不稳定性**：虽然 GRPO 相比 PPO 更稳定，但强化学习过程仍然对奖励函数的设计和超参数敏感，不当的设置可能导致模型行为退化或产生“奖励黑客”（reward hacking）现象。
  - **量化带来的性能损失**：虽然实验声称性能下降可忽略不计，但在更复杂的任务或对精度要求极高的场景中，4-bit 量化仍可能引入不可接受的误差。

实验设计与结果

实验设计

本研究设计了一系列对比实验，以系统地评估知识蒸馏（KD）和链式思考强化学习（RL-CoT）在不同场景下的有效性。

1. **英语知识蒸馏**：
   - **设置**：使用 Qwen-3B 作为教师模型，Qwen-0.5B 作为学生模型，在英语 Dolly-15k 数据集上进行训练。
   - **变量**：探索不同学习率（`1e-5` 到 `5e-4`）对蒸馏效果的影响。
   - **目标**：确定最优学习率，并验证 KD 在高资源语言（英语）上的基准性能。
2. **西班牙语知识蒸馏**：
   - **设置**：在西班牙语 Dolly 数据集上进行。首先，将教师模型（Qwen-3B）在该数据集上进行 SFT，以使其成为该领域的“专家”。
   - **对比**：比较两种学生模型（Qwen-0.5B）的性能：一个通过 SFT 训练，另一个通过 KD 训练。
   - **目标**：验证在模型先验知识有限的低资源场景下，KD 相对于 SFT 的优势。
3. **代码任务知识蒸馏**：
   - **设置**：在 BugNet（代码修复）和 PyTorrent（代码生成）两个数据集上进行。
   - **模型**：使用了 Qwen2 和 Qwen2.5 两个系列的多个模型组合进行实验。

- **目标**：评估 KD 在结构化和逻辑性要求更高的代码任务上的表现。

4. **CoT 强化学习**：
   - **设置**：在代码生成任务的基础上，对经过 KD 的学生模型使用 `codeforces-cots` 数据集和 GRPO 算法进行 RL 训练。
   - **奖励**：设计了复杂的奖励函数，激励模型生成包含特定关键词（如 "Step *", "think"）和正确逻辑的 CoT 轨迹。
   - **目标**：验证 RL-CoT 是否能显著提升小型模型在复杂推理任务上的性能。

实验结果

1. 知识蒸馏普遍优于监督微调（SFT）

在所有任务中，通过知识蒸馏（KD）训练的学生模型在 ROUGE-L 分数上均显著优于仅通过监督微调（SFT）训练的同等规模模型。特别是在西班牙语任务中，KD 模型（ROUGE-L 19.7）的表现远超 SFT 模型（17.4），并逼近了教师模型的性能（20.6），显示出在低资源场景下的巨大潜力。

2. 学习率对蒸馏效果影响显著

在英语蒸馏实验中，学习率 `5e-4` 被证明是最佳选择，其训练损失下降最快，最终 ROUGE-L 分数最高，这与原始 MiniLLM 论文的发现一致。这表明为 KD 选择合适的学习率至关重要。

3. RL-CoT 大幅提升代码推理能力

在代码生成任务中，经过 RL-CoT 增强的模型表现出最佳性能。从定性分析（见下表）可以看出，未经训练的 Base 模型无法理解指令，SFT 和 KD 模型能生成一些相关代码但逻辑混乱或不完整，而经过 RL-CoT 训练的模型能够生成结构清晰、逻辑正确的复杂代码（如 DFS 算法）。这证明了 RL-CoT 在解锁小型模型复杂推理能力方面的有效性。

4. 模型对先验知识的依赖

实验结果清晰地表明，模型的性能与其对任务领域的先验知识密切相关。在模型预训练程度较高的英语上，性能提升相对平稳。而在知识较少的西班牙语和高度结构化的代码领域，KD 和 RL-CoT 带来的性能增益则更为显著。

生成效果对比表下表展示了不同训练方法下，模型在西班牙语问答和代码生成任务上的定性表现。

任务	训练方法	生成结果示例	ROUGE-L	分析
西班牙语问答	Base (基础模型)	Cuál has a lot to offer so you’ll want to check them out.	3.1	完全错误，生成了不相关的英文。
	SFT (监督微调)	El capital de Reino Unido es la Ópera...	17.4	部分正确，但混淆了概念。
	KD (知识蒸馏)		19.7	

		La capital de Reino Unido es Nueva York.		答案错误，但句式和语义结构正确。
	Teacher (教师模型)	Londres es la capital de Reino Unido.	20.6	完全正确。
代码生成 (DFS)	Base (基础模型)	• list all visited nodes using ’ depth_first_search	8.7	仅重复了部分指令，未生成代码。
	SFT (监督微调)	def search(self, start=None, path=[], ...	34.4	生成了函数签名，但实现不完整或错误。
	KD (知识蒸馏)	def _next(x, d): x = next(x) return ...	35.5	生成了部分逻辑片段，但非完整算法。
	RL-CoT (蒸馏+强化学习)	def DFS(D= Graph ( )): ... D.add_edge(5,0) ...	37.5	生成了结构完整、逻辑正确的 DFS 算法实现。

注：上表内容根据论文中的表格和图示综合整理。

讨论与分析

1. **为什么知识蒸馏比 SFT 更有效？** 知识蒸馏的优势在于它传递了教师模型的“暗知识”（dark knowledge）。SFT 仅仅学习了“正确答案是什么”（硬目标），而 KD 学习了教师模型认为“哪些答案比较可能，哪些完全不可能”的完整概率分布（软目标）。这种分布信息包含了更丰富的类别间关系，为学生模型的训练提供了更平滑、更具信息量的梯度，从而引导其更好地泛化。
2. **GRPO 在 CoT 增强中的作用**传统的 RLHF 方法如 PPO 需要训练一个独立的奖励模型（RM）和一个价值函数（Critic），流程复杂且不稳定。GRPO 通过在一个批次内对多个生成结果进行**相对排序**来计算优势，完全摒弃了对 RM 和 Critic 的依赖。这种“群体智慧”式的优化方式更简单、更稳定，特别适合 CoT 这种奖励信号可以被明确规则（如是否包含特定思考步骤）定义的场景。
3. **对端侧部署的意义**本文最大的实践价值在于展示了一条将云端大模型能力“下放”到端侧的可行路径。通过“蒸馏-增强-量化”三步曲，可以在保持核心任务性能（甚至在专业任务上超越通用模型）的同时，将模型尺寸压缩到适合移动设备部署的级别。这对于推动 AI 在边缘计算领域的普及具有重要意义。

结论

本文成功地演示了一个高效的框架，用于将大型语言模型的能力压缩并迁移到小模型中。研究表明：

- 知识蒸馏是一种比监督微调更高效的模型压缩技术，尤其是在目标领域数据有限的情况下。
- 结合链式思考（CoT）的强化学习（特别是 GRPO 算法）能够有效增强小型模型在复杂推理任务（如编程）上的表现，弥补了因参数量减少而损失的部分能力。
- 通过进一步的训练后量化，最终生成的模型可以在内存和算力受限的端侧设备上高效运行，为大模型的广泛应用和隐私保护提供了新的解决方案。

这项工作为开发轻量级、高性能的专用 AI 模型提供了宝贵的实践经验和理论支持。

参考文献

- Gu, Y., Dong, L., Wei, F., & Huang, M. (2024). **MiniLLM: Knowledge Distillation of Large Language Models**. ICLR.
- DeepSeek-AI, et al. (2025). **Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning**. arXiv preprint arXiv:2501.12948.
- Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2020). **Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter**. arXiv preprint arXiv:1910.01108.
- Paredes La Torre, A., Flores, B., & Rodriguez, D. (2026). **Knowledge Distillation for Large Language Models**. arXiv preprint arXiv:2603.13765.[1]

参考资料

[1] Knowledge Distillation for Large Language Models

[2] Alejandro Paredes La Torre

[3] Crayon: Customized On-Device LLM via Instant Adapter Blending and Edge-Server Hybrid Inference

[4] Improving Safety Alignment via Balanced Direct Preference Optimization

[5] GitHub - verl-project/verl: verl: Volcano Engine Reinforcement ...

[6] FedPDPO: Federated Personalized Direct Preference Optimization for Large Language Model Alignment